A complementary example for performance could be supporting multi-threading: if your application does not involve (ever) sharing these containers among different threads, you can avoid making your implementation thread-safe, and save the overhead needed to create and synchronize locks.

If performance and memory usage are critical for your application, you will probably want to write a different, optimized version of this code where these fancy language features are not used; likely, you will also avoid using recursion and write explicit loops.
But, if low-level optimization is not critical, you might prefer sticking with cleaner and more maintainable code, using better interfaces and APIs, because in the long run having readable code will make your life (and the job of future team members) much easier.

3.6 Summary

- **Binary Search Trees**: this data structure offers good performances on all the typical container's methods, but only if it's kept balanced: depending on the order of insertion of its keys, however, a BST can become skewed
- **Balanced vs Unbalanced trees**: while in a balanced tree all paths from the root to leaves have logarithmic length, `log(n)` if `n` entries are stored in the tree, in a skewed tree some paths are short, and others are too long: the edge case is when an ordered sequence is added to a BST, that will then contain a single path of length `n`, *de facto* degenerating in a linked list.
- **Treap**: this is a peculiar data structure whose nodes hold 2 pieces of data: a key, and a priority; treaps are a hybrid